

Low-Level Design Document

Project Overview

This document provides a detailed overview of the Controller, Model, and Component layers of the project. It outlines the responsibilities, key methods, and interactions between these layers.

Low-Level Design Document

🎓 School Management System Modules

📁 Core Academic Modules

1. Student Information System (SIS) 📖

- Student enrollment and registration
- Student profiles and demographics
- Parent/Guardian information
- Academic records management
- Student status tracking

2. Course Management System 📖

- Course creation and setup
- Curriculum management
- Class scheduling
- Course materials distribution
- Teacher assignments
- Syllabus management

3. Examination System 📝

- Exam scheduling
- Question bank management
- Online examination tools
- Result processing
- Performance analytics
- Grade calculation

4. Gradebook Management 📊

- Grade entry and calculation
- Progress tracking
- Report card generation
- Academic performance analysis
- Grade distribution analytics
- Parent notification system

5. Attendance Management 📅

- Daily attendance tracking
- Leave management
- Absence reporting
- Attendance statistics
- Parent notification
- Attendance compliance monitoring

6. Assessment Management

- Continuous assessment tracking
- Project evaluation
- Assignment management
- Performance metrics
- Skill assessment
- Learning outcome tracking

7. Survey Management

- Student feedback collection
- Parent satisfaction surveys
- Teacher evaluation surveys
- Course evaluation
- Analytics and reporting
- Survey template management

Supporting Modules

8. Communication Module

- Internal messaging
- Parent communication
- Announcements
- Email notifications
- SMS alerts

9. Document Management

- Student documents
- Academic records
- Certificates
- Transcripts
- Digital storage

10. Reporting System

- Academic reports
- Administrative reports
- Custom report generation
- Data analytics
- Performance dashboards

11. User Management

- User roles and permissions
- Authentication
- Profile management
- Access control
- User activity tracking

Module Interactions

Module Interaction Diagram

Each module interacts with others to create a comprehensive educational management system:

Primary Interactions

📖 Detailed Module Documentation

1. Schools Module

🎮 SchoolsController

📖 Student Information System (SIS) - Technical Documentation

1. Database Structure

Core Tables

This document provides a detailed overview of the `Controller`, `Model`, and `Component` layers of the project. It outlines the responsibilities, key methods, and interactions between these layers.

Controllers

UsersController

- **Purpose**: Handles user-related actions such as registration, login, profile management, and user-specific operations.
- **Key Methods**:
 - `register()`: Handles user registration, including form display and data processing.
 - `login()`: Manages user authentication, verifying credentials, and establishing a session.
 - `logout()`: Ends the user session and redirects to the login page.
 - `profile(\$id = null)`: Displays and allows editing of user profile information.
 - `changePassword()`: Provides functionality for users to update their passwords.
 - `forgotPassword()`: Handles password recovery process, including sending reset links.
- **Interactions**:
 - **Models**: Interacts with the `User` model to perform CRUD operations on user data.
 - **Components**: Utilizes components like `Auth` for authentication, `Session` for session management, and `RequestHandler` for handling different request types.

AbsencesController

- **Purpose**: Manages student absences, including adding, editing, viewing, and deleting absence records.
- **Key Methods**:
 - `student\_daily\_absences()`: Redirects to specific absence actions based on school group.
 - `add\_leave(\$student\_id)`: Handles the addition of leave records for students.

- ``edit_leave($student_leave_id)``: Edits existing leave records.
- ``delete_leave($student_leave_id)``: Deletes leave records.
- ``students_leaves($id = null)``: Manages the list of student leaves.
- **Interactions**:
 - Interacts with models like ``StudentLeave``, ``Absence``, and ``StudentDetailsView`` to perform CRUD operations.
 - Utilizes components like ``RequestHandler`` and ``Session`` for handling requests and session data.

SchoolsController

- **Purpose**: Manages school-related functionalities, including viewing, editing, and managing school data and settings.
- **Key Methods**:
 - ``index()``: Displays a list of all schools, possibly with filtering and pagination.
 - ``view($id = null)``: Shows detailed information about a specific school.
 - ``add()``: Handles the creation of a new school, including form display and data processing.
 - ``edit($id = null)``: Allows editing of existing school information.
 - ``delete($id = null)``: Deletes a school record, typically with confirmation to prevent accidental deletion.
 - ``settings($id = null)``: Manages school-specific settings, such as academic year, semester details, and other configurations.
- **Interactions**:
 - **Models**: Interacts with the ``School`` model to perform CRUD operations on school data.
 - **Components**: Utilizes components like ``Session`` for session management and ``RequestHandler`` for handling different request types.
 - **Views**: Renders views to display school data and forms for adding/editing schools.

YearsController

- **Purpose**: Manages academic years within the application.
- **Key Methods**:
 - ``index()``: Lists all academic years.
 - ``add()``: Adds a new academic year.
 - ``edit($id = null)``: Edits an existing academic year.
 - ``delete($id = null)``: Deletes an academic year.
- **Interactions**:
 - **Models**: Interacts with the ``Year`` model for CRUD operations.

SemestersController

- **Purpose**: Manages semesters within academic years.
- **Key Methods**:
 - ``index()``: Lists all semesters.
 - ``add()``: Adds a new semester.
 - ``edit($id = null)``: Edits an existing semester.
 - ``delete($id = null)``: Deletes a semester.
- **Interactions**:
 - **Models**: Interacts with the ``Semester`` model for CRUD operations.

StagesController

- **Purpose**: Manages educational stages (e.g., primary, secondary).
- **Key Methods**:

- ``index()``: Lists all stages.
- ``add()``: Adds a new stage.
- ``edit($id = null)``: Edits an existing stage.
- ``delete($id = null)``: Deletes a stage.
- **Interactions**:
 - **Models**: Interacts with the ``Stage`` model for CRUD operations.

LevelsController

- **Purpose**: Manages educational levels within stages.
- **Key Methods**:
 - ``index()``: Lists all levels.
 - ``add()``: Adds a new level.
 - ``edit($id = null)``: Edits an existing level.
 - ``delete($id = null)``: Deletes a level.
- **Interactions**:
 - **Models**: Interacts with the ``Level`` model for CRUD operations.

SectionsController

- **Purpose**: Manages sections within levels.
- **Key Methods**:
 - ``index()``: Lists all sections.
 - ``add()``: Adds a new section.
 - ``edit($id = null)``: Edits an existing section.
 - ``delete($id = null)``: Deletes a section.
- **Interactions**:
 - **Models**: Interacts with the ``Section`` model for CRUD operations.

CoursesController

- **Purpose**: Manages all course-related operations including creation, modification, and enrollment management.
- **Key Methods**:
 - ``index()``: Lists all courses with filtering and pagination options
 - ``view($id = null)``: Displays detailed course information
 - ``add()``: Creates new course entries
 - ``edit($id = null)``: Modifies existing course information
 - ``delete($id = null)``: Removes course entries
 - ``enroll($course_id, $student_id)``: Handles student enrollment in courses
 - ``unenroll($course_id, $student_id)``: Manages student withdrawal from courses
 - ``assignTeacher($course_id, $teacher_id)``: Assigns teachers to courses
 - ``getMaterials($course_id)``: Retrieves course materials
 - ``getSchedule($course_id)``: Retrieves course schedule
- **Interactions**:
 - **Models**: Interacts with ``Course``, ``Enrollment``, ``Teacher``, ``Student``, ``CourseMaterial`` models
 - **Components**: Uses ``Auth``, ``Session``, ``RequestHandler``, ``Pagination``
 - **Events**: Triggers events for enrollment notifications and schedule updates

LecturesController

- **Purpose**: Manages lecture sessions, scheduling, and delivery
- **Key Methods**:

- ``index()`: Lists all lectures with filtering options
- ``view($id = null)`: Shows detailed lecture information
- ``add()`: Creates new lecture entries
- ``edit($id = null)`: Updates lecture information
- ``delete($id = null)`: Removes lecture entries
- ``schedule($course_id)`: Manages lecture scheduling
- ``attendance($lecture_id)`: Handles attendance tracking
- ``materials($lecture_id)`: Manages lecture materials
- ``recording($lecture_id)`: Handles lecture recordings
- ``getLiveStatus($lecture_id)`: Checks lecture live status
- ``streamLecture($lecture_id)`: Manages live lecture streaming
- **Interactions**:
 - **Models**: Interacts with ``Lecture`, ``Course`, ``Attendance`, ``LectureMaterial`, ``Recording`
- **models**
- **Components**: Uses ``Auth`, ``Session`, ``StreamHandler`, ``Calendar`
- **Events**: Triggers notifications for schedule changes and live sessions

TimeslotsController

- **Purpose**: Manages scheduling time slots for lectures and courses
- **Key Methods**:
 - ``index()`: Lists all available time slots
 - ``view($id = null)`: Shows detailed time slot information
 - ``add()`: Creates new time slot entries
 - ``edit($id = null)`: Updates time slot information
 - ``delete($id = null)`: Removes time slot entries
 - ``checkAvailability($start_time, $end_time)`: Checks slot availability
 - ``getConflicts($timeslot_id)`: Identifies scheduling conflicts
 - ``assignToLecture($timeslot_id, $lecture_id)`: Associates slot with lecture
 - ``assignToCourse($timeslot_id, $course_id)`: Associates slot with course
 - ``getScheduleMatrix()`: Generates schedule visualization
 - ``bulkCreate()`: Creates multiple time slots at once
- **Interactions**:
 - **Models**: Interacts with ``Timeslot`, ``Course`, ``Lecture`, ``Room`
 - **Components**: Uses ``Calendar`, ``Scheduler`, ``Validation`
 - **Events**: Triggers notifications for schedule changes

StudyPlanController

- **Purpose**: Manages academic study plans and curriculum mapping
- **Key Methods**:
 - ``index()`: Lists all study plans
 - ``view($id = null)`: Shows detailed study plan information
 - ``add()`: Creates new study plan
 - ``edit($id = null)`: Updates study plan information
 - ``delete($id = null)`: Removes study plan
 - ``assignCourses($plan_id)`: Associates courses with study plan
 - ``generateSequence()`: Creates course sequence recommendations
 - ``validatePrerequisites($plan_id)`: Validates prerequisite chain
 - ``checkCompletion($student_id, $plan_id)`: Tracks completion status
 - ``generateReport($plan_id)`: Creates study plan progress report
 - ``clonePlan($plan_id)`: Creates a copy of existing study plan
- **Interactions**:
 - **Models**: Interacts with ``StudyPlan`, ``Course`, ``Prerequisite`, ``StudentProgress`

- **Components**: Uses `Auth`, `Validation`, `ProgressTracker`
- **Events**: Triggers notifications for plan updates and milestone achievements

AssessmentsController

- **Purpose**: Manages all types of student assessments, evaluations, and grading
- **Key Methods**:
 - `index()`: Lists all assessments with filtering options
 - `view(\$id = null)`: Shows detailed assessment information
 - `add()`: Creates new assessment
 - `edit(\$id = null)`: Updates assessment information
 - `delete(\$id = null)`: Removes assessment
 - `grade(\$assessment\_id, \$student\_id)`: Records student grades
 - `submitResponse(\$assessment\_id)`: Handles student submissions
 - `generateRubric()`: Creates assessment rubrics
 - `calculateScores()`: Computes final scores
 - `exportResults(\$assessment\_id)`: Exports assessment results
 - `reviewSubmission(\$submission\_id)`: Reviews student submissions
 - `provideFeedback(\$submission\_id)`: Manages feedback for submissions
- **Interactions**:
 - **Models**: Interacts with `Assessment`, `Submission`, `Grade`, `Rubric`, `Feedback`
 - **Components**: Uses `Auth`, `Validation`, `GradeCalculator`, `ExportManager`
 - **Events**: Triggers notifications for submission deadlines and grade postings

SurveysController

- **Purpose**: Manages survey creation, distribution, and response collection
- **Key Methods**:
 - `index()`: Lists all surveys with filtering options
 - `view(\$id = null)`: Shows detailed survey information
 - `add()`: Creates new survey
 - `edit(\$id = null)`: Updates survey information
 - `delete(\$id = null)`: Removes survey
 - `distribute(\$survey\_id)`: Sends surveys to participants
 - `collectResponses(\$survey\_id)`: Gathers survey responses
 - `generateReport(\$survey\_id)`: Creates survey analysis reports
 - `exportData(\$survey\_id)`: Exports survey data
 - `closeSurvey(\$survey\_id)`: Ends survey collection period
 - `analyzeResults(\$survey\_id)`: Performs statistical analysis
 - `sendReminders(\$survey\_id)`: Manages reminder notifications
- **Interactions**:
 - **Models**: Interacts with `Survey`, `SurveyQuestion`, `SurveyResponse`, `SurveyAnalytics`
 - **Components**: Uses `Auth`, `Notification`, `Analytics`, `ExportManager`
 - **Events**: Triggers notifications for survey distribution and completion

CertificatesController

- **Purpose**: Manages student certificates, including generation, verification, and distribution
- **Key Methods**:
 - `index()`: Lists all certificates with filtering options
 - `view(\$id = null)`: Shows detailed certificate information
 - `add()`: Creates new certificate template
 - `edit(\$id = null)`: Updates certificate information
 - `delete(\$id = null)`: Removes certificate

- ``generate($student_id)``: Generates certificate for student
- ``verify($certificate_id)``: Verifies certificate authenticity
- ``download($certificate_id)``: Handles certificate downloads
- ``bulkGenerate($course_id)``: Generates certificates for multiple students
- ``revoke($certificate_id)``: Revokes issued certificate
- ``template($id = null)``: Manages certificate templates
- ``sign($certificate_id)``: Handles digital signatures
- **Interactions**:
 - **Models**: Interacts with ``Certificate``, ``Template``, ``Student``, ``Course``, ``Signature``
 - **Components**: Uses ``Auth``, ``PDFGenerator``, ``DigitalSignature``, ``QRGenerator``
 - **Events**: Triggers notifications for certificate issuance and verification

AssignmentsController

- **Purpose**: Manages course assignments, submissions, and grading
- **Key Methods**:
 - ``index()``: Lists all assignments with filtering options
 - ``view($id = null)``: Shows detailed assignment information
 - ``add()``: Creates new assignment
 - ``edit($id = null)``: Updates assignment information
 - ``delete($id = null)``: Removes assignment
 - ``submit($assignment_id)``: Handles student submissions
 - ``grade($submission_id)``: Records grades for submissions
 - ``extend($assignment_id)``: Manages deadline extensions
 - ``checkPlagiarism($assignment_id)``: Runs plagiarism detection
 - ``downloadSubmissions($assignment_id)``: Bulk downloads submissions
 - ``provideFeedback($submission_id)``: Manages submission feedback
 - ``generateReport($assignment_id)``: Creates assignment reports
- **Interactions**:
 - **Models**: Interacts with ``Assignment``, ``Submission``, ``Grade``, ``Extension``, ``PlagiarismReport``
 - **Components**: Uses ``Auth``, ``FileHandler``, ``PlagiarismDetector``, ``NotificationManager``
 - **Events**: Triggers notifications for deadlines, submissions, and grades

PreparationsController

- **Purpose**: Manages lesson preparations and teaching materials
- **Key Methods**:
 - ``index()``: Lists all lesson preparations
 - ``view($id = null)``: Shows detailed preparation information
 - ``add()``: Creates new lesson preparation
 - ``edit($id = null)``: Updates preparation information
 - ``delete($id = null)``: Removes preparation
 - ``attachMaterials($preparation_id)``: Handles teaching material attachments
 - ``shareWithTeachers($preparation_id)``: Shares preparations with other teachers
 - ``clone($preparation_id)``: Creates a copy of existing preparation
 - ``generatePDF($preparation_id)``: Exports preparation as PDF
 - ``reviewPreparation($preparation_id)``: Handles preparation review process
 - ``approvePreparation($preparation_id)``: Manages approval workflow
 - ``linkToLecture($preparation_id, $lecture_id)``: Associates preparation with lectures
- **Interactions**:
 - **Models**: Interacts with ``Preparation``, ``PreparationMaterial``, ``Teacher``, ``Lecture``
 - **Components**: Uses ``Auth``, ``FileHandler``, ``PDFGenerator``, ``ReviewSystem``
 - **Events**: Triggers notifications for review requests and approvals

DigitalRepositoryController

- **Purpose**: Manages digital learning resources and institutional repository
- **Key Methods**:
 - `index()`: Lists all repository items with filtering options
 - `view($id = null)`: Shows detailed item information
 - `add()`: Adds new repository item
 - `edit($id = null)`: Updates repository item information
 - `delete($id = null)`: Removes repository item
 - `search($query)`: Performs advanced repository search
 - `download($item_id)`: Handles item downloads
 - `categorize($item_id)`: Manages item categorization
 - `tagItem($item_id)`: Handles item tagging
 - `trackUsage($item_id)`: Monitors resource usage
 - `shareResource($item_id)`: Manages resource sharing
 - `generateCitation($item_id)`: Creates academic citations
- **Interactions**:
 - **Models**: Interacts with `RepositoryItem`, `Category`, `Tag`, `Citation`, `Usage`
 - **Components**: Uses `Auth`, `Search`, `FileHandler`, `CitationGenerator`
 - **Events**: Triggers notifications for new uploads and shares

WorkflowApprovalsController

- **Purpose**: Manages approval workflows and processes across the system
- **Key Methods**:
 - `index()`: Lists all pending and completed approvals
 - `view($id = null)`: Shows detailed approval information
 - `create()`: Creates new approval workflow
 - `process($approval_id)`: Processes approval request
 - `approve($approval_id)`: Approves workflow step
 - `reject($approval_id)`: Rejects workflow step
 - `delegate($approval_id, $user_id)`: Delegates approval authority
 - `track($workflow_id)`: Tracks workflow progress
 - `escalate($approval_id)`: Handles approval escalations
 - `remind($approval_id)`: Sends reminder notifications
 - `generateReport($workflow_id)`: Creates workflow reports
 - `auditTrail($workflow_id)`: Maintains approval history
- **Interactions**:
 - **Models**: Interacts with `WorkflowApproval`, `ApprovalStep`, `ApprovalHistory`, `User`
 - **Components**: Uses `Auth`, `Notification`, `AuditLogger`, `WorkflowEngine`
 - **Events**: Triggers notifications for approval requests and decisions

AnswersController

- **Purpose**: Manages student answers and responses to assessments and questions
- **Key Methods**:
 - `index()`: Lists all answers with filtering options
 - `view($id = null)`: Shows detailed answer information
 - `add()`: Creates new answer entry
 - `edit($id = null)`: Updates answer information
 - `delete($id = null)`: Removes answer entry
 - `grade($answer_id)`: Handles answer grading
 - `checkPlagiarism($answer_id)`: Detects potential plagiarism
 - `provideFeedback($answer_id)`: Manages answer feedback

- `bulkGrade(\$assessment\_id)`: Handles bulk grading
- `exportAnswers(\$assessment\_id)`: Exports answer data
- `validateAnswer(\$answer\_id)`: Validates answer content
- `trackHistory(\$answer\_id)`: Monitors answer changes
- **Interactions**:
 - **Models**: Interacts with `Answer`, `Question`, `Assessment`, `Grade`, `Feedback`
 - **Components**: Uses `Auth`, `ValidationHandler`, `PlagiarismChecker`
 - **Events**: Triggers notifications for grading and feedback

Models

User

- **Purpose**: Represents the user entity in the application, encapsulating user data and providing methods for data manipulation.
- **Key Responsibilities**:
 - **Data Validation**: Ensures user data integrity through validation rules.
 - **Associations**: Defines relationships with other models, such as `Profile`, `Role`, and `Permission`.
 - **CRUD Operations**: Provides methods for creating, reading, updating, and deleting user records.
 - **Authentication**: Supports user authentication processes, including password hashing and verification.
- **Interactions**:
 - **Database**: Interacts with the database to persist user data.
 - **Controllers**: Provides data and methods to controllers like `UsersController` for user management.
 - **Components**: May interact with components for additional functionalities like email verification or password recovery.

Absence

- **Purpose**: Represents student absence records.
- **Key Responsibilities**:
 - Define relationships with other models like `Student` and `StudentLeave`.
 - Provide methods for querying and manipulating absence data.

Year

- **Purpose**: Represents an academic year.
- **Key Responsibilities**:
 - Manage year-specific data.
 - Define relationships with `Semester`.

Semester

- **Purpose**: Represents a semester within an academic year.
- **Key Responsibilities**:
 - Manage semester-specific data.
 - Define relationships with `Year` and `Stage`.

Stage

- **Purpose**: Represents an educational stage.
- **Key Responsibilities**:
 - Manage stage-specific data.
 - Define relationships with `Level`.

Level

- **Purpose**: Represents an educational level within a stage.
- **Key Responsibilities**:
 - Manage level-specific data.
 - Define relationships with `Stage` and `Section`.

Section

- **Purpose**: Represents a section within a level.
- **Key Responsibilities**:
 - Manage section-specific data.
 - Define relationships with `Level`.

Course

- **Purpose**: Represents the course entity and manages course-related data operations
- **Key Responsibilities**:
 - **Data Management**:
 - Course information storage and retrieval
 - Enrollment tracking
 - Schedule management
 - Materials organization
 - **Relationships**:
 - `belongsTo`: `School`, `Department`
 - `hasMany`: `Enrollment`, `CourseMaterial`, `Assignment`
 - `belongsToMany`: `Student`, `Teacher`
 - **Validation Rules**:
 - Course code uniqueness
 - Capacity limits
 - Schedule conflicts
 - Prerequisites validation
 - **Key Methods**:
 - `getAvailableSlots()`: Calculates remaining enrollment capacity
 - `checkPrerequisites(\$student\_id)`: Validates student prerequisites
 - `getEnrolledStudents()`: Retrieves list of enrolled students
 - `getAssignedTeachers()`: Retrieves assigned teaching staff
 - `getMaterialsList()`: Retrieves course materials
 - `getScheduleConflicts()`: Checks for scheduling conflicts

CourseEnrollment

- **Purpose**: Manages the relationship between students and courses
- **Key Responsibilities**:
 - Track enrollment status
 - Manage enrollment dates
 - Handle withdrawal processes
 - Store enrollment history

- **Relationships**:
 - `belongsTo`: `Course`, `Student`
 - `hasMany`: `EnrollmentHistory`
- **Validation Rules**:
 - Enrollment period validation
 - Capacity checks
 - Duplicate enrollment prevention
 - Prerequisite verification

CourseMaterial

- **Purpose**: Manages course-related resources and materials
- **Key Responsibilities**:
 - Store material metadata
 - Handle file attachments
 - Manage access permissions
 - Track usage statistics
- **Relationships**:
 - `belongsTo`: `Course`
 - `hasMany`: `MaterialAccess`
- **Validation Rules**:
 - File size limits
 - Supported format validation
 - Access permission checks
 - Version control

Lecture

- **Purpose**: Represents individual lecture sessions and manages lecture-related data
- **Key Responsibilities**:
 - **Data Management**:
 - Lecture information storage
 - Schedule management
 - Attendance tracking
 - Recording management
 - **Relationships**:
 - `belongsTo`: `Course`, `Teacher`
 - `hasMany`: `Attendance`, `LectureMaterial`, `Recording`
 - `hasOne`: `LiveSession`
 - **Validation Rules**:
 - Schedule conflict prevention
 - Duration constraints
 - Room capacity validation
 - Technical requirements verification
 - **Key Methods**:
 - `isLive()`: Checks if lecture is currently live
 - `getAttendanceStats()`: Retrieves attendance statistics
 - `getMaterialsUsed()`: Lists materials used in lecture
 - `getRecordingStatus()`: Checks recording availability
 - `checkRoomAvailability()`: Verifies room availability
 - `notifyParticipants()`: Sends notifications to attendees

LectureAttendance

- **Purpose**: Manages attendance records for lectures
- **Key Responsibilities**:
 - Track student attendance
 - Generate attendance reports
 - Handle late entries
 - Manage excuse documentation
- **Relationships**:
 - `belongsTo`: `Lecture`, `Student`
 - `hasMany`: `AttendanceLog`
- **Validation Rules**:
 - Time stamp validation
 - Duplicate entry prevention
 - Late entry policies
 - Excuse verification

LectureMaterial

- **Purpose**: Manages materials used during lectures
- **Key Responsibilities**:
 - Store presentation files
 - Handle handouts
 - Manage reference materials
 - Track material usage
- **Relationships**:
 - `belongsTo`: `Lecture`
 - `hasMany`: `MaterialAccess`
- **Validation Rules**:
 - File format restrictions
 - Size limitations
 - Access permissions
 - Version control

LectureRecording

- **Purpose**: Manages lecture recordings and playback
- **Key Responsibilities**:
 - Handle recording storage
 - Manage playback access
 - Track viewing statistics
 - Control distribution
- **Relationships**:
 - `belongsTo`: `Lecture`
 - `hasMany`: `ViewLog`
- **Validation Rules**:
 - Storage capacity checks
 - Format compatibility
 - Access restrictions
 - Retention policies

Timeslot

- **Purpose**: Represents time slots for scheduling purposes
- **Key Responsibilities**:
 - **Data Management**:

- Time slot information storage
- Availability tracking
- Conflict detection
- Schedule optimization
- **Relationships**:
 - `belongsTo`: `Room`, `Teacher`
 - `hasMany`: `Lecture`, `Course`
 - `hasOne`: `RecurringPattern`
- **Validation Rules**:
 - Time range validation
 - Overlap prevention
 - Working hours compliance
 - Room availability check
- **Key Methods**:
 - `isAvailable()`: Checks slot availability
 - `hasConflicts()`: Detects scheduling conflicts
 - `getAssignedResources()`: Lists assigned rooms/teachers
 - `getNextAvailable()`: Finds next available slot
 - `generateRecurringSlots()`: Creates recurring time slots
 - `optimizeSchedule()`: Suggests optimal scheduling

RecurringPattern

- **Purpose**: Manages recurring time slot patterns
- **Key Responsibilities**:
 - Define repetition rules
 - Handle exceptions
 - Manage pattern modifications
 - Track pattern instances
- **Relationships**:
 - `belongsTo`: `Timeslot`
 - `hasMany`: `PatternException`
- **Validation Rules**:
 - Pattern validity checks
 - Exception handling rules
 - Date range constraints
 - Frequency limitations

TimeslotAllocation

- **Purpose**: Manages the allocation of time slots to resources
- **Key Responsibilities**:
 - Track resource assignments
 - Handle allocation conflicts
 - Manage capacity utilization
 - Monitor usage patterns
- **Relationships**:
 - `belongsTo`: `Timeslot`, `Resource`
 - `hasMany`: `AllocationLog`
- **Validation Rules**:
 - Resource availability
 - Capacity constraints
 - Allocation duration
 - Priority handling

StudyPlan

- **Purpose**: Represents academic study plans and curriculum structures
- **Key Responsibilities**:
 - **Data Management**:
 - Plan structure storage
 - Course sequence management
 - Prerequisite tracking
 - Progress monitoring
 - **Relationships**:
 - `belongsTo`: `Department`, `Program`
 - `hasMany`: `Course`, `Prerequisite`, `Milestone`
 - `belongsToMany`: `Student`
 - **Validation Rules**:
 - Credit hour requirements
 - Course sequence validity
 - Prerequisite chain integrity
 - Graduation requirements
- **Key Methods**:
 - `validateSequence()`: Validates course sequence logic
 - `calculateProgress(\$student\_id)`: Computes completion percentage
 - `getPrerequisiteChain(\$course\_id)`: Retrieves prerequisite hierarchy
 - `suggestAlternatives(\$course\_id)`: Suggests alternative courses
 - `checkGraduationEligibility(\$student\_id)`: Verifies graduation requirements
 - `generateStudyPath()`: Creates optimized study path

StudyPlanMilestone

- **Purpose**: Manages key achievements and checkpoints in study plans
- **Key Responsibilities**:
 - Track milestone completion
 - Monitor progress requirements
 - Handle milestone dependencies
 - Generate achievement reports
- **Relationships**:
 - `belongsTo`: `StudyPlan`
 - `hasMany`: `MilestoneRequirement`, `Achievement`
- **Validation Rules**:
 - Milestone sequence validation
 - Requirement completion checks
 - Dependency validation
 - Time constraints

StudyPlanProgress

- **Purpose**: Tracks student progress through study plans
- **Key Responsibilities**:
 - Monitor course completion
 - Track credit hours
 - Calculate GPA
 - Generate progress reports
- **Relationships**:
 - `belongsTo`: `StudyPlan`, `Student`

- `hasMany`: `CourseGrade`, `ProgressLog`
- **Validation Rules**:
 - Grade validation
 - Credit hour tracking
 - Progress calculation
 - Status updates

Assessment

- **Purpose**: Represents assessment entities and manages assessment-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Assessment information storage
 - Question management
 - Rubric organization
 - Score calculation
 - **Relationships**:
 - `belongsTo`: `Course`, `Teacher`
 - `hasMany`: `Question`, `Submission`, `Grade`, `Rubric`
 - `belongsToMany`: `Student`
 - **Validation Rules**:
 - Due date validation
 - Score range verification
 - Weight distribution
 - Time limit constraints
- **Key Methods**:
 - `calculateTotalScore()`: Computes total possible score
 - `checkSubmissionStatus(\$student\_id)`: Verifies submission status
 - `generateGradeReport()`: Creates detailed grade reports
 - `validateRubric()`: Ensures rubric completeness
 - `checkPlagiarism()`: Detects potential plagiarism
 - `generateStatistics()`: Computes assessment statistics

AssessmentSubmission

- **Purpose**: Manages student submissions for assessments
- **Key Responsibilities**:
 - Track submission timing
 - Store student responses
 - Handle file uploads
 - Manage submission versions
- **Relationships**:
 - `belongsTo`: `Assessment`, `Student`
 - `hasMany`: `SubmissionFile`, `Grade`, `Feedback`
- **Validation Rules**:
 - Submission deadline compliance
 - File type restrictions
 - Size limitations
 - Required fields validation

AssessmentRubric

- **Purpose**: Manages grading criteria and scoring guidelines
- **Key Responsibilities**:

- Define scoring criteria
- Set performance levels
- Manage scoring weights
- Track rubric versions
- **Relationships**:
 - `belongsTo`: `Assessment`
 - `hasMany`: `RubricCriteria`, `RubricLevel`
- **Validation Rules**:
 - Weight distribution
 - Criteria completeness
 - Score range validity
 - Level progression

AssessmentFeedback

- **Purpose**: Manages feedback for student submissions
- **Key Responsibilities**:
 - Store feedback comments
 - Track feedback history
 - Manage improvement suggestions
 - Handle feedback attachments
- **Relationships**:
 - `belongsTo`: `Submission`, `Teacher`
 - `hasMany`: `FeedbackAttachment`
- **Validation Rules**:
 - Feedback timing
 - Content appropriateness
 - Attachment restrictions
 - Version control

Survey

- **Purpose**: Represents survey entities and manages survey-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Survey structure storage
 - Question organization
 - Response collection
 - Analytics processing
 - **Relationships**:
 - `belongsTo`: `Course`, `Teacher`
 - `hasMany`: `SurveyQuestion`, `SurveyResponse`, `SurveyAnalytics`
 - `belongsToMany`: `Student`, `Participant`
 - **Validation Rules**:
 - Survey period validation
 - Question completeness
 - Response validation
 - Participant access control
- **Key Methods**:
 - `calculateResponseRate()`: Computes participation percentage
 - `generateAnalytics()`: Creates statistical analysis
 - `checkCompletionStatus(\$participant\_id)`: Verifies completion
 - `exportResponses()`: Prepares data export
 - `validateQuestions()`: Ensures survey integrity

- `generateSummaryReport()`: Creates overview reports

SurveyQuestion

- **Purpose**: Manages individual survey questions and their properties
- **Key Responsibilities**:
 - Store question content
 - Handle question types
 - Manage answer options
 - Track question dependencies
- **Relationships**:
 - `belongsTo`: `Survey`
 - `hasMany`: `QuestionOption`, `QuestionResponse`
- **Validation Rules**:
 - Question format validation
 - Option completeness
 - Dependency validation
 - Required field checks

SurveyResponse

- **Purpose**: Manages participant responses to surveys
- **Key Responsibilities**:
 - Store response data
 - Track completion status
 - Handle partial responses
 - Manage response versions
- **Relationships**:
 - `belongsTo`: `Survey`, `Participant`
 - `hasMany`: `QuestionResponse`
- **Validation Rules**:
 - Response completeness
 - Time stamp validation
 - Required answers
 - Format validation

SurveyAnalytics

- **Purpose**: Handles survey data analysis and reporting
- **Key Responsibilities**:
 - Process response data
 - Generate statistics
 - Create visualizations
 - Track trends
- **Relationships**:
 - `belongsTo`: `Survey`
 - `hasMany`: `AnalyticsReport`, `Visualization`
- **Validation Rules**:
 - Data integrity
 - Statistical validity
 - Report accuracy
 - Visualization parameters

Certificate

- **Purpose**: Represents certificate entities and manages certificate-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Certificate information storage
 - Template management
 - Verification handling
 - Digital signature processing
 - **Relationships**:
 - `belongsTo`: `Student`, `Course`, `Template`
 - `hasMany`: `Signature`, `VerificationLog`
 - `hasOne`: `QRCode`
 - **Validation Rules**:
 - Certificate uniqueness
 - Template validity
 - Signature requirements
 - Expiration rules
- **Key Methods**:
 - `generateCertificateNumber()`: Creates unique certificate ID
 - `validateSignatures()`: Verifies digital signatures
 - `checkRevocationStatus()`: Checks if certificate is revoked
 - `generateQRCode()`: Creates verification QR code
 - `createPDF()`: Generates PDF version
 - `trackUsage()`: Monitors certificate usage

CertificateTemplate

- **Purpose**: Manages certificate design templates and layouts
- **Key Responsibilities**:
 - Store template designs
 - Handle template versions
 - Manage design elements
 - Control template access
- **Relationships**:
 - `belongsTo`: `School`, `Department`
 - `hasMany`: `Certificate`, `TemplateElement`
- **Validation Rules**:
 - Design requirements
 - Element positioning
 - Size constraints
 - Format compatibility

CertificateSignature

- **Purpose**: Manages digital signatures for certificates
- **Key Responsibilities**:
 - Store signature data
 - Verify signature authenticity
 - Track signature chain
 - Handle signature revocation
- **Relationships**:
 - `belongsTo`: `Certificate`, `User`
 - `hasMany`: `SignatureLog`
- **Validation Rules**:

- Signature validity
- Authority verification
- Timestamp accuracy
- Chain integrity

Assignment

- ****Purpose****: Represents assignment entities and manages assignment-related operations
- ****Key Responsibilities****:
 - ****Data Management****:
 - Assignment information storage
 - Submission tracking
 - Deadline management
 - Grade calculation
 - ****Relationships****:
 - ``belongsTo``: ``Course``, ``Teacher``
 - ``hasMany``: ``Submission``, ``Grade``, ``Extension``
 - ``hasOne``: ``PlagiarismSettings``
 - ****Validation Rules****:
 - Deadline validation
 - File type restrictions
 - Submission limits
 - Grade scale validation
- ****Key Methods****:
 - ``isOverdue()``: Checks submission deadline status
 - ``calculateFinalGrade()``: Computes final grades
 - ``checkSubmissionStatus($student_id)``: Verifies submission status
 - ``generateSubmissionReport()``: Creates submission summary
 - ``validateSubmissionFormat()``: Checks submission requirements
 - ``trackPlagiarism()``: Monitors academic integrity

AssignmentSubmission

- ****Purpose****: Manages student assignment submissions
- ****Key Responsibilities****:
 - Track submission timing
 - Store submission files
 - Handle version control
 - Manage submission status
- ****Relationships****:
 - ``belongsTo``: ``Assignment``, ``Student``
 - ``hasMany``: ``SubmissionFile``, ``SubmissionHistory``
- ****Validation Rules****:
 - Deadline compliance
 - File format validation
 - Size restrictions
 - Version tracking

AssignmentExtension

- ****Purpose****: Manages deadline extensions for assignments
- ****Key Responsibilities****:
 - Store extension details
 - Track extension requests

- Handle approval process
- Monitor extended deadlines
- **Relationships**:
 - `belongsTo`: `Assignment`, `Student`
 - `hasOne`: `ApprovalStatus`
- **Validation Rules**:
 - Extension duration limits
 - Request justification
 - Approval requirements
 - History tracking

AssignmentGrade

- **Purpose**: Manages grading for assignment submissions
- **Key Responsibilities**:
 - Store grade information
 - Calculate weighted scores
 - Track grading history
 - Manage grade appeals
- **Relationships**:
 - `belongsTo`: `Submission`, `Teacher`
 - `hasMany`: `GradeComponent`, `Appeal`
- **Validation Rules**:
 - Grade range validation
 - Component weighting
 - Grading scheme compliance
 - Appeal process rules

Preparation

- **Purpose**: Represents lesson preparation entities and manages preparation-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Preparation content storage
 - Material organization
 - Review tracking
 - Version control
 - **Relationships**:
 - `belongsTo`: `Teacher`, `Course`
 - `hasMany`: `PreparationMaterial`, `ReviewComment`
 - `belongsToMany`: `Lecture`
 - **Validation Rules**:
 - Content completeness
 - Material references
 - Review requirements
 - Version tracking
- **Key Methods**:
 - `isApproved()`: Checks approval status
 - `getReviewStatus()`: Retrieves review information
 - `generateContentOutline()`: Creates content structure
 - `validateMaterials()`: Verifies attached materials
 - `trackUsage()`: Monitors preparation usage
 - `createRevision()`: Handles version management

PreparationMaterial

- **Purpose**: Manages materials associated with lesson preparations
- **Key Responsibilities**:
 - Store material files
 - Track material usage
 - Handle material versions
 - Manage access rights
- **Relationships**:
 - `belongsTo`: `Preparation`
 - `hasMany`: `MaterialVersion`, `Usage`
- **Validation Rules**:
 - File type validation
 - Size restrictions
 - Version control
 - Access permissions

PreparationReview

- **Purpose**: Manages the review process for lesson preparations
- **Key Responsibilities**:
 - Track review status
 - Store review comments
 - Handle approval workflow
 - Maintain review history
- **Relationships**:
 - `belongsTo`: `Preparation`, `Reviewer`
 - `hasMany`: `ReviewComment`, `ReviewHistory`
- **Validation Rules**:
 - Review cycle validation
 - Comment requirements
 - Approval criteria
 - Timeline tracking

RepositoryItem

- **Purpose**: Represents digital repository items and manages resource-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Resource metadata storage
 - File management
 - Version control
 - Access tracking
 - **Relationships**:
 - `belongsTo`: `User`, `Category`
 - `hasMany`: `Version`, `Usage`, `Citation`
 - `belongsToMany`: `Tag`, `Collection`
 - **Validation Rules**:
 - Metadata completeness
 - File integrity
 - Format compatibility
 - License validation
- **Key Methods**:

- ``generateMetadata()``: Creates resource metadata
- ``checkAccessRights()``: Verifies access permissions
- ``trackDownloads()``: Monitors download statistics
- ``createVersion()``: Handles version management
- ``generateDOI()``: Creates Digital Object Identifier
- ``validateFormat()``: Checks file format compliance

RepositoryCategory

- ****Purpose****: Manages repository resource categorization
- ****Key Responsibilities****:
 - Organize resources
 - Manage category hierarchy
 - Handle cross-references
 - Track category usage
- ****Relationships****:
 - ``hasMany``: ``RepositoryItem``, ``Subcategory``
 - ``belongsTo``: ``ParentCategory``
- ****Validation Rules****:
 - Category naming
 - Hierarchy depth
 - Classification accuracy
 - Usage tracking

RepositoryUsage

- ****Purpose****: Tracks usage of repository resources
- ****Key Responsibilities****:
 - Monitor downloads
 - Track citations
 - Generate statistics
 - Analyze usage patterns
- ****Relationships****:
 - ``belongsTo``: ``RepositoryItem``, ``User``
 - ``hasMany``: ``UsageLog``, ``Analytics``
- ****Validation Rules****:
 - Access logging
 - Usage metrics
 - Statistical accuracy
 - Privacy compliance

RepositoryCitation

- ****Purpose****: Manages academic citations for repository items
- ****Key Responsibilities****:
 - Generate citations
 - Track citations
 - Support multiple formats
 - Manage references
- ****Relationships****:
 - ``belongsTo``: ``RepositoryItem``
 - ``hasMany``: ``CitationFormat``, ``Reference``
- ****Validation Rules****:
 - Format accuracy

- Reference validity
- Style compliance
- Version tracking

WorkflowApproval

- **Purpose**: Represents approval workflows and manages approval-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Workflow configuration storage
 - Approval chain management
 - Status tracking
 - History maintenance
 - **Relationships**:
 - `belongsTo`: `User`, `Department`
 - `hasMany`: `ApprovalStep`, `ApprovalHistory`
 - `belongsToMany`: `Approver`
 - **Validation Rules**:
 - Workflow completeness
 - Authority validation
 - Sequence integrity
 - Deadline compliance
- **Key Methods**:
 - `getCurrentStep()`: Retrieves current approval step
 - `validateApprovalChain()`: Verifies approval sequence
 - `checkApprovalStatus()`: Checks overall status
 - `calculateTurnaroundTime()`: Measures approval duration
 - `generateAuditTrail()`: Creates approval audit log
 - `enforceDeadlines()`: Manages approval deadlines

ApprovalStep

- **Purpose**: Manages individual steps in approval workflows
- **Key Responsibilities**:
 - Track step status
 - Handle step transitions
 - Manage approver assignments
 - Monitor step deadlines
- **Relationships**:
 - `belongsTo`: `WorkflowApproval`, `Approver`
 - `hasMany`: `StepAction`, `StepNotification`
- **Validation Rules**:
 - Step sequence validation
 - Approver authorization
 - Deadline requirements
 - Action validation

ApprovalHistory

- **Purpose**: Tracks approval workflow history and changes
- **Key Responsibilities**:
 - Record approval actions
 - Store decision comments
 - Track timeline

- Maintain audit trail
- **Relationships**:
 - `belongsTo`: `WorkflowApproval`, `User`
 - `hasMany`: `HistoryAttachment`, `Comment`
- **Validation Rules**:
 - Action logging
 - Timeline accuracy
 - Comment requirements
 - Document retention

Answer

- **Purpose**: Represents student answers and manages answer-related operations
- **Key Responsibilities**:
 - **Data Management**:
 - Answer content storage
 - Version tracking
 - Grade management
 - Feedback handling
 - **Relationships**:
 - `belongsTo`: `Student`, `Question`, `Assessment`
 - `hasMany`: `Grade`, `Feedback`, `AnswerHistory`
 - `hasOne`: `PlagiarismReport`
 - **Validation Rules**:
 - Content validation
 - Word count limits
 - Format requirements
 - Submission timing
- **Key Methods**:
 - `isGraded()`: Checks grading status
 - `calculateScore()`: Computes answer score
 - `checkPlagiarism()`: Runs plagiarism detection
 - `getFeedbackHistory()`: Retrieves feedback timeline
 - `validateFormat()`: Checks answer format
 - `trackRevisions()`: Manages answer versions

AnswerGrade

- **Purpose**: Manages grading for individual answers
- **Key Responsibilities**:
 - Store grade data
 - Calculate partial scores
 - Track grading history
 - Handle grade appeals
- **Relationships**:
 - `belongsTo`: `Answer`, `Teacher`
 - `hasMany`: `GradeComponent`, `Appeal`
- **Validation Rules**:
 - Score range validation
 - Grading criteria
 - Feedback requirements
 - Appeal process

AnswerFeedback

- ****Purpose****: Manages feedback for student answers
- ****Key Responsibilities****:
 - Store feedback content
 - Track feedback versions
 - Handle improvement suggestions
 - Manage feedback attachments
- ****Relationships****:
 - `belongsTo`: `Answer`, `Teacher`
 - `hasMany`: `FeedbackAttachment`
- ****Validation Rules****:
 - Content requirements
 - Timing constraints
 - Format validation
 - Attachment limits

AnswerHistory

- ****Purpose****: Tracks changes and versions of answers
- ****Key Responsibilities****:
 - Store version history
 - Track modifications
 - Manage revision logs
 - Handle version comparison
- ****Relationships****:
 - `belongsTo`: `Answer`
 - `hasMany`: `HistoryLog`
- ****Validation Rules****:
 - Version tracking
 - Change logging
 - Timeline accuracy
 - Storage limits

Components

RequestHandler

- ****Purpose****: Handles HTTP requests and responses.
- ****Key Responsibilities****:
 - Determine request types (e.g., AJAX, JSON).
 - Manage response formats and headers.

Session

- ****Purpose****: Manages user session data.
- ****Key Responsibilities****:
 - Store and retrieve session variables.
 - Handle session lifecycle events.

Auth

- ****Purpose****: Provides authentication and authorization functionalities.

- **Key Responsibilities**:
 - Manage user login and logout processes.
 - Check user permissions and roles.

StreamHandler

- **Purpose**: Manages live lecture streaming
- **Key Responsibilities**:
 - Handle streaming protocols
 - Manage streaming quality
 - Handle streaming errors

Calendar

- **Purpose**: Manages calendar events and schedules
- **Key Responsibilities**:
 - Handle calendar integration
 - Manage event creation and editing
 - Handle event conflicts

Interactions and Data Flow

- **Controllers** interact with **Models** to perform data operations and with **Components** to handle requests, sessions, and authentication.
- **Models** encapsulate the data logic and provide an interface for data manipulation.
- **Components** offer reusable functionalities that can be leveraged across different controllers.

Conclusion

This document outlines the core components, models, and controllers within the project, detailing their responsibilities and interactions. This structure ensures a modular and maintainable codebase, facilitating future development and enhancements.

Low-Level Design Documentation

System Components

1. Code Parser

- Identifies code sections requiring modification
- Maintains code structure integrity
- Handles multiple programming languages

2. Format Handler

- Processes markdown formatting
- Manages code block syntax
- Handles file path annotations

3. Language Processor

- Detects user input language
- Manages response translations
- Preserves technical accuracy

Component Interactions

Code Update Flow

1. Receive user input
2. Parse code context
3. Generate modifications
4. Format response
5. Add file paths and language IDs

File Creation Flow

1. Receive new file request
2. Generate complete file content
3. Apply proper formatting
4. Include file path and language ID

Technical Specifications

Response Format

- Markdown-based structure
- Code block formatting
- File path notation

Code Block Structure

Advanced System Architecture

Data Flow Architecture

1. Request Processing Layer

- Input validation and sanitization
- Request type identification
- Priority queue management
- Rate limiting implementation

2. Core Processing Engine

- Code analysis algorithms
- Context awareness system
- Pattern recognition
- Code optimization suggestions

3. Response Generation Layer

- Template management
- Dynamic content generation
- Response optimization
- Format validation

Security Implementation

1. Input Validation

- Code injection prevention
- Path traversal protection
- Special character handling
- Input size limitations

2. Access Control

- Permission management
- Rate limiting
- API key validation
- Session management

Performance Optimization

1. Caching Strategy

- Response caching
- Template caching
- Frequently used patterns
- Cache invalidation rules

2. Resource Management

- Memory allocation
- Thread management
- Connection pooling
- Resource cleanup

Error Handling Framework

1. Error Categories

- Syntax errors
- Runtime errors
- Logic errors
- System errors

2. Error Recovery

- Graceful degradation
- Automatic retry mechanism
- Fallback options
- Error logging and monitoring

Monitoring and Logging

1. System Metrics

- Response time tracking
- Resource utilization
- Error rate monitoring
- Usage patterns

2. Audit Trail

- User actions
- System modifications
- Security events
- Performance bottlenecks

Integration Interfaces

1. External API Integration

- REST API endpoints
- WebSocket connections
- Authentication protocols
- Rate limiting

2. Plugin Architecture

- Plugin loading mechanism
- Extension points
- Version compatibility
- Plugin isolation

Testing Framework

1. Unit Testing

- Component isolation
- Mock objects
- Test coverage
- Assertion frameworks

2. Integration Testing

- Component interaction
- End-to-end scenarios
- Load testing
- Security testing

Deployment Architecture

1. Container Strategy

- Docker configuration
- Container orchestration
- Service discovery
- Load balancing

2. CI/CD Pipeline

- Automated builds
- Test automation
- Deployment automation
- Rollback procedures

Documentation Standards

1. Code Documentation

- Inline comments
- API documentation
- Function signatures
- Usage examples

2. System Documentation

- Architecture diagrams
- Component interaction
- Configuration guide
- Troubleshooting guide

Maintenance Procedures

1. Regular Maintenance

- Log rotation
- Cache clearing
- Database optimization
- Security updates

2. Emergency Procedures

- System recovery
- Data backup
- Incident response
- Communication protocol

System Architecture Diagrams

Core System Structure

```
```mermaid
graph TD
```

```
...
```

## ## Code Structure

```
```mermaid
```

```
graph TD
```

```
  subgraph Core
```

```
    App[App Core]
```

```
    Config[Configuration]
```

```
    Router[Router]
```

```
  end
```

```
  subgraph Controllers
```

```
    AC[Academic Controllers]
```

```
    UC[User Controllers]
```

```
    MC[Management Controllers]
```

```
  end
```

```
  subgraph Models
```

```
    AM[Academic Models]
```

```
    UM[User Models]
```

```
    MM[Management Models]
```

```
  end
```

```
  subgraph Components
```

```
    Auth[Authentication]
```

```
    Session[Session Management]
```

```
    FileHandler[File Handler]
```

```
    Notification[Notification System]
```

```
  end
```

```
  App --> Router
```

```
  Router --> Controllers
```

Controllers --> Models
Controllers --> Components
Models --> Components

```
subgraph Academic_Controllers
  AC --> CoursesController
  AC --> LecturesController
  AC --> AssessmentsController
  AC --> AssignmentsController
  AC --> SurveysController
end
```

```
subgraph Management_Controllers
  MC --> WorkflowController
  MC --> CertificatesController
  MC --> RepositoryController
  MC --> PreparationsController
end
```

```
subgraph Academic_Models
  AM --> Course
  AM --> Lecture
  AM --> Assessment
  AM --> Assignment
  AM --> Survey
end
```

```
subgraph Management_Models
  MM --> Workflow
  MM --> Certificate
  MM --> Repository
  MM --> Preparation
end
```

...

Directory Structure

...

```
app/
├── Controllers/
│   ├── Academic/
│   │   ├── CoursesController.php
│   │   ├── LecturesController.php
│   │   ├── AssessmentsController.php
│   │   └── AssignmentsController.php
│   ├── Management/
│   │   ├── WorkflowController.php
│   │   ├── CertificatesController.php
│   │   └── RepositoryController.php
│   └── User/
```



```

|   |   |   UsersController.php
|   |   |   RolesController.php
|   |   |
|   |   |--- Models/
|   |   |   |   |   Academic/
|   |   |   |   |   |   Course.php
|   |   |   |   |   |   Lecture.php
|   |   |   |   |   |   Assessment.php
|   |   |   |   |   |   Assignment.php
|   |   |   |   |   |
|   |   |   |   |   |--- Management/
|   |   |   |   |   |   |   Workflow.php
|   |   |   |   |   |   |   Certificate.php
|   |   |   |   |   |   |   Repository.php
|   |   |   |   |   |   |
|   |   |   |   |   |   |--- User/
|   |   |   |   |   |   |   User.php
|   |   |   |   |   |   |   Role.php
|   |   |   |   |   |   |
|   |   |   |   |   |--- Components/
|   |   |   |   |   |   |   Auth/
|   |   |   |   |   |   |   Session/
|   |   |   |   |   |   |   FileHandler/
|   |   |   |   |   |   |   Notification/
|   |   |   |   |   |   |
|   |   |   |   |   |--- Config/
|   |   |   |   |   |   |   app.php
|   |   |   |   |   |   |   database.php
|   |   |   |   |   |   |   routes.php
|   |   |   |   |   |   |
|   |   |   |   |   |--- Views/
|   |   |   |   |   |   |   Academic/
|   |   |   |   |   |   |   Management/
|   |   |   |   |   |   |   User/
|   |   |   |   |   |   |
|   |   |   |   |   |--- ...

```

Key Relationships

```

```mermaid
erDiagram
 COURSE ||--o{ LECTURE : contains
 COURSE ||--o{ ASSESSMENT : includes
 COURSE ||--o{ ASSIGNMENT : has
 COURSE ||--o{ ENROLLMENT : manages

 STUDENT ||--o{ ENROLLMENT : has
 STUDENT ||--o{ SUBMISSION : makes
 STUDENT ||--o{ ATTENDANCE : records

 TEACHER ||--o{ COURSE : teaches
 TEACHER ||--o{ PREPARATION : creates
 TEACHER ||--o{ ASSESSMENT : manages

```

LECTURE ||--o{ ATTENDANCE : tracks  
LECTURE ||--o{ MATERIAL : uses

WORKFLOW ||--o{ APPROVAL : manages  
WORKFLOW ||--o{ HISTORY : tracks

CERTIFICATE ||--o{ SIGNATURE : contains  
CERTIFICATE ||--o{ VERIFICATION : allows